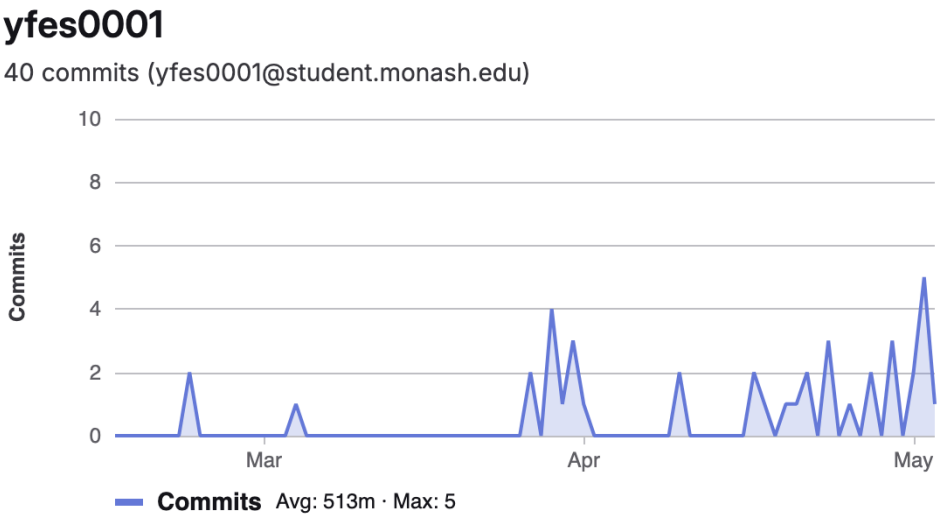
FIT3077 Semester 1 2025

Sprint 3 (Individual)

Yokabit Fesshaye 32591675

Contributor Analytics **2**
Sprint 3 Overview **2**
Updated UML Class Diagram **3**
Design Rationale **4**
New Class/Interface Addition 4
Code Reassignment 6
Alignment with Human Value: Leaderboard Extension 7
Justification on Chosen Solution 8
Discarded Design Choice 8
Design Patterns 11
Reflection on Sprint 2 Design **13**
Instructions for Executable **15**

Contributor Analytics



Sprint 3 Overview

For the following Sprint, all three required extensions have been implemented. The extensions are as follows:

1. God Card Extension: Triton

In this extension, the god card Triton was introduced. Triton's unique ability enhances player strategy by allowing additional movement options that interact dynamically with the game board. Implementing Triton involved defining its special powers within the game logic, ensuring seamless integration with the existing turn and action system, and updating the UI to display Triton's abilities clearly to players.

2. Player Timer System

A timer system was developed to add time pressure and strategic pacing to the gameplay. Each player receives a fixed total amount of time of 5 minutes (which can be extended to allow users to select the time limit in future sprint iterations) to think and execute their turns. The timer pauses automatically at the end of a player's turn and starts counting down for the next player. 5 minutes is all the time a player is given. If a player's time runs out, they immediately lose the game. The timers are displayed in the user interface, allowing all players to monitor their remaining time continuously.

3. Leaderboard Implementation

A fully functional leaderboard was implemented to enhance player engagement and provide persistent tracking of game outcomes. The leaderboard stores player names and win counts externally in a text file, enabling data to be stored across game sessions. The UI was designed to present the leaderboard clearly and intuitively, located in the main menu of the application, improving user experience by allowing players to view rankings and track their progress over time. This extension deepens the competitive aspect of the game and encourages replayability.

Updated UML Class Diagram

The updated final UML class diagram is located in the git repository, which can be found with the following link:
https://git.infotech.monash.edu/FIT3077/fit3077-s1-2025/assignment-individual/yfes0001/-/blob/main/sprint_3/docs/sprint_3_docs/Sprint_3_UML_Diagram.png?ref_type=heads

During the sprint 3 iteration, there were significant changes to the sprint 2 iteration. For clarity, the changes can be seen through the aid of the colours in the diagram:

- Pink: indicates the new class additions for the sprint 3 extensions
- Purple: indicates the changes to existing classes from sprint 2. These adjustments involve method changes, addition or deletion of variables and reassigning responsibilities.
- Green: indicates the new class additions that have been implemented to accommodate for the changes made to the sprint 2 iterations.

These changes were important as the sprint 2 implementation wasn't necessarily following object oriented principles and some classes contained a significant amount of responsibilities. Overall, sprint 2 implementation did not have completed game functionalities that followed the Santorini game board gameplay. In order to implement sprint 3 and have a completed game, the changes shown in the UML class diagram were implemented.

Design Rationale

New Class/Interface Addition

For the implementation of Sprint 3, there were many classes added whether it was to integrate the new extensions, or adjust the previous Sprint and add complete functionality to the existing application.

The significant class additions and justifications are as follows in the table below:

Classes/Interfaces and their Relationships	Justification
ActionList Relationships: Used by God (superclass for Demeter, Artemis, Triton) to store actions. Accessed by TurnManager to execute actions sequentially during a player's turn.	ActionList is a class that holds a list of Action objects such as MoveAction, BuildAction, DemeterAction, ArtemisAction and TritonAction, for a player's god card, defining the sequence of actions a worker can perform in each player's turn. No existing class in the provided codebase prior to its addition was designed to manage a sequence of actions for god cards. It was previously handled by GamePanel which was a code smell. Adding this to Player or God would mix concerns, as Player manages player state and God defines god-specific behavior. If it were handled by an existing class, it would've become overly complex and violate single responsibility principles. This class provides a flexible way to define and execute variable action sequences for different gods, supporting extensibility for new god powers, such as Triton in Sprint 3. Without it, each god would need hardcoded action logic, reducing modularity and code duplication. Not reassignable, as it's a core component for handling god-specific action sequences. ActionClass is essential for managing the sequence of actions for each god card, enabling varied gameplay mechanics, making it a core component in the system, hence it cannot be reassigned.
TurnManager Relationships: Interacts with GamePanel to access the board Player to track current player TurnTimer to switch timers WinCondition and LossCondition to check game outcomes. Uses ActionList to execute god-specific actions.	TurnManager class manages turn-based gameplay, handling player switching, action execution, worker selection, skipping actions, and win/loss condition checks. Previously, the methods and logic within the TurnManager class was located in GamePanel. The decision to remove it from GamePanel class was so that GamePanel could handle initialisation of the game and UI and board rendering, and not turn logic. Delegating turn management to Player class would overburden it, as players represent state, not game flow. Hence why it has its separate class, combining turn logic with either would lead to bloated, less maintainable code.

	This class is vital for the implementation of the game as it centralises turn logic, ensuring consistent handling of player actions and structured turn progression, worker selection, and game state transitions, integrating action execution, timer management, and win/loss checks. It coordinates with TurnTimer and god actions for a cohesive gameplay loop. Reassigning the class to another existing class would not be feasible as it is core to the game and primary mechanism for the turn-based nature of the Santorini game.
LeaderBoard Relationships: Used by TitleScreenPanel to display scores, WinCondition and LossCondition to update scores on game end Independent of other gameplay classes like GamePanel or TurnManager.	This class could not exist in an existing class since no Sprint 2 class handled score tracking or file I/O for persistent leaderboards. Hence why it is a new addition to the updated UML diagram. Adding to GamePanel or Player would mix concerns, as they focus on gameplay and player state, respectively. LeaderBoard is an important class as it supports the third extension implemented in Sprint 3 by tracking and rewarding wins, enhancing replayability, competition, and adds depth to the overall game. Persistent storage ensures scores transfer across sessions and can be built on in future games. No prior class had the functionality present in LeaderBoard so reassigning to another class is not possible. Existing classes are also not suited for score management or file I/O. Adding to GamePanel would violate separation of concerns, as it is focused on rendering and game state. Adding LeaderBoard to the TitleScreenPanel class would also violate separation of concerns as it is only responsible for viewing all elements that belong on the title screen, which is why the two classes are separate but connected as seen in the UML class diagram.
TitleScreenPanel Relationships: Uses LeaderBoard to display scores. Instantiates GamePanel and Player when starting the game.	TitleScreenPanel is responsible for displaying the main menu with the game title, leaderboard display, and start game/exit buttons. The title screen also contains an additional feature which is getting the player’s names. The names of each player is what's used to store the player’s win count in the leaderboard and each time that same player plays the game and enters the same name, their score in the leaderboard will be updated if they end up winning. Other than that, the title screen triggers the initialisation of the game board, players, god cards and workers the moment the button start game is clicked. TitleScreenPanel would not be able to be reassigned to any existing class since all other classes concern themselves with essential functionality to gameplay, not menu or leaderboard display functionality. No other class in Sprint 2 managed pre-game UI or initialisation. Combining menu logic with gameplay would reduce modularity in the software design.

	The TitleScreenPanel class provides a user-friendly entry point, separating menu logic from gameplay and enhances user experience by offering leaderboard access and clear game controls.
TurnTimer Relationships: Integrated with GamePanel for game state BoardView for timer labels, TurnManager to switch timers LossCondition to handle timeouts. Uses Swing's Timer for countdown.	Since Sprint 2 had no timing mechanism,. Adding to GamePanel or TurnManager would overload them, as they focus on rendering and turn logic, respectively. The TurnTimer class would also require a decent amount of logic and structure so separating its responsibilities into a separate class was logical as it is distinct from rendering or turn management. Separating timer logic keeps TurnManager focused on turn flow. This class is the most important class for the timer extension implemented in Sprint 3, introducing time pressure, enhancing gameplay challenge. It adds a time-based challenge, aligning with the extension's goal of increasing gameplay intensity. A dedicated class ensures clean separation of concerns, making it easier to integrate, debug and enhance.
TritonAction/ArtemisAction/ DemeterAction Relationships: Subclasses of MoveAction (TritonAction, ArtemisAction) or BuildAction (DemeterAction). Contained in ActionList for each god (Triton, Artemis, Demeter). Used by TurnManager during action execution.	The three god action classes are new additions to the system design, with TritonAction being the new addition based on the Sprint 3 extension requirement. These classes are specialised Action subclasses implementing god-specific powers (Triton: move again on perimeter tiles; Artemis: move again one more time but not to initial tile; Demeter: build again, not on same tile). Generic MoveAction and BuildAction lack god-specific logic hence why these 3 classes were implemented, to implement the god actions. Hard coding these behaviors in God, GamePanel or TurnManager would reduce extensibility for new gods, increase code smells and make it harder to debug. These classes enable unique god powers, a core feature of the game. Each action encapsulates specific rules ensuring modularity and ease of adding new gods. It could be possible to reassign the god actions as methods in their respective god classes, however for this implementation it was best to create a new class for each god action. There is some repeated code logic that could be reassigned to Action class, and the god classes could implement a method that calls other methods from action class to implement their respective god-specific actions.

Code Reassignment

To accommodate for the extensions for Sprint 3, such as the introduction of the Triton god card, score tracking with leaderboard and time-based gameplay with TurnTimer, a significant amount of

responsibilities from Sprint 2 were reassigned to maintain modularity and support new features. Specifically, move and build action logic was extracted from GamePanel and encapsulated into their respective action classes, MoveAction and BuildAction, and god-specific actions like TritonAction, ArtemisAction and DemeterAction in their respective god classes to enable extensibility for new god powers and streamline action execution within TurnManager.

Turn management logic was also reassigned from GamePanel to TurnManager, centralising turn-based gameplay mechanics and coordinating with the new TurnTimer for time constraints. These reassignments ensured GamePanel focused solely on board state and rendering, improving code organisation and supporting Sprint 3 enhanced gameplay features and extensions,

Additionally, UI-related responsibilities, previously handled by GamePanel, were moved to BoardView which was formerly TileManager, which now manages comprehensive UI rendering beyond just tile images, including workers, timer labels, highlighting valid moves, and buttons, hence the name change for the class.

Alignment with Human Value: Leaderboard Extension

The third extension of the Santorini game centers on the human value of Achievement, as defined in Schwartz's 58 human values. Achievement emphasises personal success through demonstrating competence and reaching goals. This value is particularly important in games, as it motivates players to strive for excellence, enhances replayability, and creates a compelling reason to return and improve. By recognising players' accomplishments, games that align with this value can encourage a strong sense of purpose, challenge, and satisfaction, especially among competitive players who thrive on measurable success.

To align with this value, a leaderboard with a scoring system was implemented in Sprint 3. The new LeaderBoard class tracks player performance by awarding points to the winners. This system provides an opportunity to demonstrate skill and improve standing. The leaderboard is displayed on the title screen, allowing players to view their current rankings before starting a new game. This visibility creates a sense of achievement and encourages continued engagement, as players are motivated to outperform others and climb the rankings.

From the player's perspective, the value of Achievement is manifested through immediate and visible feedback. Winning a game leads to a score increase, which is reflected on the leaderboard, reinforcing a feeling of success. The competitive aspect of seeing other players' scores further pushes individuals to improve. The integration of this extension can be directly observed in three areas, the TitleScreenPanel, where the leaderboard is shown, the LeaderBoard class, which handles file I/O and score management, and the WinCondition and LossCondition classes, which update scores based on game outcomes. Additionally, the GamePanel has been modified to trigger score updates when a game ends, ensuring accurate and timely feedback.

Overall, this extension supports the value of Achievement by rewarding success with visible results such as scores and ranking, making players feel competent, accomplished, and motivated to continue playing.

Justification on Chosen Solution

The solution direction for Sprint 3 builds upon the foundational work from Sprint 2's implementation of the Santorini game. The primary architectural focus has been on modularity, extensibility, and user engagement, leading to the use of Model-View-Controller (MVC) design pattern. Since Java Swing was chosen as the UI framework for this project, it naturally complements the MVC approach by providing clear mechanisms for separating user interface rendering, game logic, and data management.

MVC design divides the application into three core components, Model, which manages the data and game state, View, which handles all user interface rendering and visual representation, and Controller, which processes input and controls interactions between the Model and the View. This separation allows for clearer responsibility among classes, enabling better maintainability, scalability, and testing. It also supports the principles of Object-Oriented Programming (OOP), such as encapsulation and single responsibility, which are critical in managing game features.

In Sprint 3 implementation, MVC was incorporated by ensuring that the codebase is organised into well-defined packages to isolate concerns, game logic is managed through components like GamePanel, WinCondition, LossCondition, and TurnManager, UI rendering is handled by view components such as BoardView, SidePanel, and TitleScreenPanel and data elements, such as Player, Worker, and Action. The LeaderBoard class, a model component responsible for managing score data, interacts with the WinCondition and LossCondition classes to update scores based on outcomes. Meanwhile, the TitleScreenPanel dynamically displays this data to the user. Because of the clean separation provided by MVC, these features were added without disrupting existing components, demonstrating the extensibility and robustness of the chosen architecture.

Ultimately, applying the MVC pattern in this project enabled structured growth, reduced code coupling, and simplified debugging. It also provided a scalable foundation for future enhancements, such as additional game modes.

Discarded Design Choice

An event-driven architecture was considered as an alternative design approach for Sprint 3. This architecture uses an event bus to manage communication between components, allowing publishers to broadcast events and subscribers to react to them. While this method can significantly reduce coupling and enhance modularity, it was ultimately not adopted for this project due to the increased complexity it introduces.

For a relatively small-scale game like this, where component interactions are limited and predictable, the added requirements of setting up an event system would outweigh its benefits. The existing direct communication and method calls between components are straightforward, easier to trace, and more suitable for debugging and iteration in a game of this scale. Additionally, Java's Observer pattern, already implemented in the TurnTimer to notify listeners of timeouts, provided the necessary event-driven behavior without requiring a full event bus infrastructure. An event-driven architecture may be more appropriate for larger or more dynamic systems, but in this case, it was unnecessary and would have complicated development.

Class Responsibility Collaborator Cards (CRC)

Triton CRC Card:

Represents the Triton god card, which extends the base God class to implement the special rule where a worker can move again after landing on a perimeter space tile. This class also contains a nested class that implements the specification action for the Triton god card which then gets used by the Triton class.

The Triton class is important to the gameplay of Santorini as it sets up the game mechanics specific to Triton. The class is then used by the GodCardAssigner class to randomly assign a player god cards, Triton being a possible god card assigned to a player.

Class Name: Triton	
Responsibilities: - Define Triton god card with special move behavior and attributes. - Sets up the action sequence for a player's turn: MoveAction, TritonAction, and BuildAction. - Supplies metadata for the Triton card, such as its description for display in the game UI and the file path to its image for rendering in the game interface. - Contains the TritonAction class which is responsible for creating the custom god power action.	Collaborators: - Worker: Triton interacts with Worker to track and update worker positions during moves, particularly to check if a worker lands on a perimeter tile. - GamePanel: Provides access to the game board and its state - MoveAction: Handles the standard movement of a worker to an adjacent tile or across different building levels. - BuildAction: Handles the basic construction of a building level on a tile that the player selects. - TritonAction: A custom action class that implements Triton's special rule, checking if a worker lands on a perimeter tile and granting an extra move if so. If granted an extra move, it adjusts TurnManager to allow for the move and if not, it skips the action.

In the early stages of design, Triton's unique ability was planned to be implemented as an additional method within the base class rather than as a dedicated subclass. This approach used a boolean flag, isOnPerimeter, to check whether the ability could be activated. At that point, perimeter checking would have been one of Triton class' responsibilities. However, since other actions like MoveAction and BuildAction were already encapsulated in their own classes, it became more consistent and manageable to implement TritonAction as a subclass. This allowed for cleaner integration with the action list and the methods responsible for executing each action.

To align better with the single-responsibility principle, the perimeter checking logic was later moved into a separate helper method, isOnPerimeter, and handled directly within TritonAction. This class now encapsulates the logic for determining whether a given tile or worker position lies on the

perimeter of the board. TritonAction now simply calls this helper method and Triton class is responsible for creating the god card and its actions.

TurnTimer CRC Card:
The TurnTimer class manages a countdown mechanism for each player, enforcing time constraints within a match. It tracks the remaining time for both players and switches control between them as turns alternate. If a player's timer reaches zero, the TurnTimer triggers a loss condition.

This class also updates the UI to reflect remaining time in real time. TurnTimer adds a strategic pressure element to gameplay and integrates with the game's core loss handling system to support a fair time-bound play.

Class Name: TurnTimer	
Responsibilities: - Track each player's remaining total game time. - Decrement the active player's timer during their turn using a regular interval. - Update the corresponding JLabel UI component to display the current remaining time. - Trigger a loss condition (due to timeout) through LossCondition when a player's timer reaches zero. - Switch active timers between players at the end of a turn.	Collaborators: - GamePanel: Provides access to the overall game state and UI context. - JLabel: Displays the remaining time for each player in the game UI. - Timer: A Java utility used to execute timed events for countdown functionality. - LossCondition: Receives notification from TurnTimer to handle timeout-based defeat.

TurnTimer was introduced as a new feature to incorporate time-based gameplay constraints as required by the extension brief. Since the LossCondition class is responsible for the different UI outputs based on the different game outcomes, it was decided to implement a method in LossCondition that handles losing due to timeout cases. This method notifies the game state which player has lost/won and this information is then later used by the TurnManager class to decide if the game is over or not.

By implementing the TurnTimer in this structure, it then becomes solely responsible for timing logic and interactions, whereas LossCondition manages the consequences of a timeout, improving modularity and testability.

LeaderBoard CRC Card:
The LeaderBoard class manages a high score system for the Santorini game. It tracks player names and their corresponding win counts, maintaining an in-memory map and syncing it with a file (leaderboard.txt).

This class ensures that only the top 10 scores are saved and displayed, keeping the leaderboard concise and competitive. It validates player names to avoid invalid entries and handles file I/O with appropriate error handling. The formatted leaderboard content is used for display in the game's UI.

Class Name: LeaderBoard	
Responsibilities: - Manage player scores using a Map<String, Integer> where names are mapped to their win counts. - Add new players or update existing player scores based on game results. - Load existing scores from leaderboard.txt into memory during initialisation. - Sort scores in descending order and save only the top 10 to the file. - Provide a formatted string of the top scores for display in the UI. - Handle file read/write operations for display.	Collaborators: - Map<String, Integer>: Stores player names (lowercased) and their win counts. - PrintWriter: Writes sorted scores to the leaderboard file. - BufferedReader: Reads scores from the leaderboard file. - File: Handles file operations for saving/loading the leaderboard. - FileWriter: Facilitates writing to the file. - FileReader: Facilitates reading from the file.

The LeaderBoard class was designed to add a persistent, competitive element to gameplay. Isolating the extension into its own class improves separation of concerns. The choice to limit entries to the top 10 keeps the leaderboard clean and focused. The overall design is straightforward and concise as it only deals with loading and saving game scores in a text file. The methods in the class are reusable in other parts of the software as well such as addScores which is called each time a player wins/loses.

There was an idea to make the leaderboard data accessible to other parts of the software such as the GodCardAssigner class. If the data was accessible to that class, then a rule that each time a player wins 3 times (has a score divisible by 3), they won't get assigned a god card and have to play the game without any powers could have been implemented, adding difficulty and fairness to the game.

Design Patterns

Template Method Pattern:
In the sprint 3 design, a template method pattern was applied to enforce a consistent structure across all action types while still allowing flexibility in behaviour. This pattern was ideal for classes that follow common algorithmic structure but implement certain steps differently, which is exactly what occurs when creating the different god cards and adding the Triton god card extension, each having the basic move and build actions and structure, but with an addition of their own unique abilities.

The abstract Action class defines the template for executing a player's in-game action, providing abstract methods such as execute and onTileClick. These methods represent steps in a predefined

structure such as selecting a tile, validating a move/build and updating the game state, but the exact detail of those steps are left to be defined by subclasses like MoveAction, BuildAction, TritonAction, DemeterAction, and ArtemisAction.
Each of these subclasses customises the behavior of the inherited methods while preserving the overall structure defined by the base Action class. This ensures that all actions follow a consistent execution flow while allowing gods with special abilities to introduce unique logic, such as Demeter building twice, or Triton checking if a worker is on a perimeter tile before repeating a move.

The Template Method pattern was used to enforce a standard sequence of steps across all player actions while allowing specific god abilities to inject their own logic where needed. This approach significantly reduces code duplication and makes the behavior of the system more predictable, which was found to be of importance when debugging and adding new features. For example, if a player wasn't able to move again after initially moving on a perimeter space tile with the Triton god card, you would navigate to the TritonAction class to fix the issue.
Rather than duplicating the logic for common checks such as tile selection validation or action validation, this logic is kept in the base class, ActionClass, and only the variable parts are implemented by each subclass, using the other logic as methods. The TurnManager can then execute a player's action in a consistent format and method, simply by executing each action in a god card's action list.

Benefits of Template Method Pattern:

- Consistency: All actions follow a shared execution structure, minimising bugs caused by inconsistent behavior.
- Extensibility: New god abilities can be added simply by extending a basic action and implementing the necessary custom logic.
  Maintainability: Shared logic is centralised in the abstract Action class, avoiding duplication across subclasses.
- Separation of concerns: The base classes handle generic control flow, while subclasses focus on specific logic.

The Template Method pattern was an effective choice given the nature of the turn-based game, where different actions share a common process but vary in implementation details. It provided the necessary structure to handle complex, god-specific rules while maintaining a clean and modular codebase.

Reflection on Sprint 2 Design

Incorporating the Sprint 3 extensions into the existing Sprint 2 codebase proved to be highly challenging and time-consuming due to fundamental issues in the original design and implementation. The Sprint 2 code suffered from multiple design flaws and code smells, making it not suited for extensibility. A prominent example was the GamePanel class, which had accumulated an unreasonable number of responsibilities, violating the Single Responsibility Principle (SRP). It was responsible not only for initialising the game and rendering the UI but also for managing game logic such as worker movement, building, turn switching, and win/loss conditions. This tight coupling of logic and UI created a brittle architecture that made it difficult to isolate or extend specific behaviors.

As a result, the first step in Sprint 3 was a substantial refactor of the codebase to create a more modular and extensible structure. Many responsibilities were extracted from GamePanel into a new TurnManager class, which now handles turn switching, action execution, worker selection, and god action skipping. Although TurnManager still carries a notable set of responsibilities, this reorganisation significantly improved cohesion and maintainability. Ideally, further decomposition into smaller, more specialised classes would better adhere to SRP and reduce the presence of long parameter lists, something to aim for in future iterations.

Difficulty of Triton God Card Implementation:
The implementation of the Triton god card was the most challenging of the three Sprint 3 extensions. This difficulty stemmed primarily from the lack of foundational support for god card functionality and action handling in the Sprint 2 codebase. While an inheritance structure for god cards was present and helpful, there was no clear design or implementation for how god-specific actions should be managed. As a result, significant refactoring was required to introduce a flexible and extensible structure for handling these actions. The GamePanel class, which had grown overly complex and violated the Single Responsibility Principle, contained much of the action logic and had to be decomposed. A new ActionList class and abstract Action subclasses were created to support polymorphic god actions. Once this structure was established, implementing TritonAction became relatively straightforward. The successful implementation of Triton now provides a clear and scalable pattern for implementing additional god cards in future iterations.

Difficulty of Timer Implementation:
The TurnTimer class itself was relatively straightforward to implement, thanks to Java Swing's built-in timer utilities. However, integrating the timer into the existing game flow presented a challenge due to architectural limitations inherited from Sprint 2. Much of the turn management logic was previously embedded in the GamePanel, contributing to bloated code and a lack of modularity. To resolve this, the system had to be refactored to separate turn logic into a dedicated TurnManager class, following the principles of MVC and SRP (Single Responsibility Principle). Once this decomposition was complete, the integration of the TurnTimer became much more manageable, as TurnManager could now directly coordinate time-based turn behavior. This restructuring not only improved maintainability but also clarified responsibility distribution, making future enhancements easier to implement.

Difficulty of Leaderboard Implementation:
The leaderboard extension was the easiest of the three to implement. Unlike the Triton and timer features, the leaderboard had no pre-existing or conflicting components in the Sprint 2 codebase, allowing for a clean implementation. A new LeaderBoard class was developed with clear separation of concerns, handling score storage, file I/O, and leaderboard display logic. The use of good OOP and MVC principles ensured smooth integration, LeaderBoard handled all score-related logic internally, exposing only necessary methods such as add and display. Minor refactoring was needed in the WinCondition and LossCondition classes to support score updates, but these changes were minimal and well-contained. The fact that rendering was handled separately in the TitleScreenPanel also made debugging easier, as issues with displaying the leaderboard could be traced to the view layer without impacting the model. Overall, this extension benefited from both clean design and low system dependency, making it efficient to develop and test.

If I were to go back and redesign the Sprint 2 implementation, I would change a significant portion of the codebase, especially the overall architecture and class responsibilities. Much of the difficulty in implementing Sprint 3 extensions stemmed from code smells, poor modularity, and unclear separation of concerns that were present from the beginning. These issues were largely the result of limited understanding and application of object-oriented principles, design patterns, and software architecture best practices at the time.

In Sprint 2, design decisions were made reactively rather than strategically, leading to classes like GamePanel becoming god classes and taking on too many responsibilities. This not only violated the Single Responsibility Principle but also made it difficult to introduce new functionality without extensive refactoring. For example, the lack of a well-defined structure for god card actions or turn handling created significant rework during Sprint 3, when those features needed to be extended.
If given the chance to start again, I would adopt a more design-first approach, investing more time during the early stages to plan class responsibilities and system interactions. I would aim to implement a clear MVC architecture from the beginning, with strict boundaries between the model, view, and controller layers. Incorporating design patterns from the start would have made the system more extensible and adaptable.

To improve future practice, I intend to make better use of UML diagrams and CRC cards during planning stages, to visualise responsibilities and interactions before diving into code. Additionally, applying refactoring techniques continuously would help maintain a clean codebase throughout development. Ultimately, Sprint 2 served as an important learning experience, highlighting the real cost of poor initial design. By reflecting critically on these challenges, I now have a clearer roadmap for building more maintainable, scalable, and well-structured software in future projects.

Instructions for Executable

The executable can be run simply by clicking on the .jar file in the directory.
It can run on any device (tested on Intel Mac, M4 MAC and Windows) and does not require any IDE or third-party libraries to be installed.
Instructions to create an executable JAR file from the source code are as follows. Keep in mind that this method only works in IntelliJ IDE.

Download JDK

1. Install JDK21 on whichever machine that will be running the software.
    - Linux: [Download JDK21] (https://www.oracle.com/au/java/technologies/downloads/#java21)
    - macOS: [Download JDK21] (https://www.oracle.com/au/java/technologies/downloads/#jdk21-mac)
    - Windows: [Download JDK21] (https://www.oracle.com/au/java/technologies/downloads/#jdk21-windows)
2. Complete the installation of the Java JDK by following the steps of the installer.

SDK Set up

1. Set up the SDK for the file by going to:
    - "File -> Project Structure..." (Shortcut: "Ctrl + Alt + Shift + S") - Click on the "Project" Tab.
2. Under project, click on the drop-down bar next to the sub-heading SDK.
3. Click on "+ ADD SDK" and then "JDK...", find the directory of the downloaded JDK and click on the whole file to add the JDK to the project.
4. Click on "Apply" then "OK" to finish the SDK set up.

Build Executable

1. Set up the SDK for the file by going to:
    - "File -> Project Structure..." (Shortcut: "Ctrl + Alt + Shift + S")
2. Click on the "Artifacts" Tab.
3. On the top left of the window, a "+" symbol will be present, add a new artifact by clicking this.
4. Click on the JAR title.
5. Then click on the "From modules with dependencies..." title, a new window will appear.
6. Next to the title "Main Class:" click on the folder icon and either search for the main class or select it from the “Project” menu. Click OK after selecting the main class.
7. In the window now, you will see all the details for your artifact, feel free to rename the jar file as you please. For this project, I have renamed the jar file as “SantoriniGame.jar”.
8. Click on "Apply" then "OK" to finish making an artifact set up.
9. Navigate to the "Build" tab on the top of your IntelliJ window.
10. Once it has been found, click on the tab and find the "Build Artifacts..." and select it.
11. A pop-up will appear, click build.
12. An executable has been created, it will be located in the out folder by default:
    - FIT3077/out/artifacts/FIT3077_jar/FIT3077.jar

    However, for this project, the jar file is located in:
    - FIT3077/sprint_3/SantoriniGame.jar

Run Executable

Option 1:

1. Locate the jar file and double click to open the executable.

Option 2:

1. Change directory in the terminal to where the jar file is being kept.
2. In the terminal write:
    - java -jar SantoriniGame.jar

    This should open the executable jar file as required.